ELI — Simulations, Plots and Graphs (2D/3D/4D)

ELI v2.5.35

November 2026

Contents

Simulations, dynamics, plots & graphs — 2D, 3D and 4D **1**
- How it works (so the claims are grounded, not magic) . . . 1
- 2D — charts, graphs, plots . . . 2
- 3D — surfaces, meshes, fields . . . 2
- 4D — dynamics, animation, things that change over time . . . 2
- What you get back . . . 2
- Honest limits . . . 2
- One-line examples to try . . . 3

Simulations, dynamics, plots & graphs — 2D, 3D and 4D

ELI can **build things that compute and draw** — from a quick bar chart to a time-evolving physics simulation. It does this the honest way: it **writes the maths/simulation code, runs it in a sandbox, and gives you back the figure or animation** (fixing the code itself if it errors). Nothing is faked — if ELI shows you a plot, real code produced it on your machine.

How it works (so the claims are grounded, not magic)

Three real pieces combine:

1. **The coding agent** (`eli/coding/agent.py`) — plans → writes → **runs** → tests → repairs. It doesn't hand you untested code; it executes it and checks the result.
2. **The sandbox** (`eli/coding/sandbox.py`) — runs that code in a clean temp directory with a **non-interactive matplotlib backend** already configured, so plots render headlessly and save to a file with no setup.
3. **The scientific stack** (the `analysis` extra, bundled in the installer): `numpy`, `scipy`, `matplotlib` (+ 3D via `mpl_toolkits.mplot3d`), `plotly`, `pandas`, `meshio`. This is the ceiling of what ELI can build — and it's a high one: the same tools professional scientists and engineers use.

So "the most advanced simulations ELI can make" = **anything expressible in Python's scientific ecosystem**, written and run for you.

2D — charts, graphs, plots

Just describe it:

- *"plot this data as a line chart"* · *"bar graph of these sales figures"*
- *"histogram of these values"* · *"scatter plot of x vs y with a trend line"*
- *"draw the function $y = x^2 - 3x + 2$ from −5 to 5"*
- *"heatmap of this matrix"* · *"pie chart of these percentages"*

Backed by **matplotlib** (publication-quality static images) and **plotly** (interactive, zoomable HTML charts).

3D — surfaces, meshes, fields

- *"show a 3D surface of $z = \sin(x)\cdot\cos(y)$"*
- *"plot this point cloud in 3D"* · *"3D scatter of these coordinates"*
- *"render this STL/OBJ/VTK mesh and report its bounds, volume and triangle count"* (via **meshio**)
- *"draw a vector field"* · *"contour plot of this 2D function"*

Backed by **mplot3d** (static 3D) and **plotly** (rotate/zoom in 3D).

4D — dynamics, animation, things that change over time

"4D" here means the **3 spatial dimensions plus time** — a simulation you can watch evolve. ELI writes the update loop, integrates the maths, and renders frames into an animation (GIF/MP4) or an interactive time-slider figure.

- **Physics:** *"simulate a pendulum and animate it"* · *"model projectile motion with air resistance"* · *"animate a bouncing ball"* · *"double pendulum chaos"*
- **Systems / ODEs:** *"solve and plot the Lorenz attractor"* · *"simulate predator- prey (Lotka–Volterra) over time"* · *"integrate this differential equation"* — powered by **scipy** solvers (`solve_ivp`, `odeint`).
- **Growth / agents:** *"model a population growing logistically for 100 steps"* · *"cellular automaton — Game of Life, animated"* · *"random walk in 2D over time"*.
- **Waves / fields:** *"animate a wave propagating"* · *"heat diffusion across a plate over time"* (a simple PDE on a grid).

Backed by **numpy** (the arrays/maths), **scipy** (the solvers — the "dynamics"), and **matplotlib.animation** / **plotly** (the moving picture).

What you get back

- A **saved figure** (PNG for static, GIF/MP4 for animation, HTML for interactive) written to ELI's artifacts, which ELI can open/show you.
- The **code** it wrote, if you want to keep, tweak, or learn from it (open it in ELI's built-in editor — see `blueprints/ide_guide.md`).
- A plain-English summary of what the simulation shows.

Honest limits

- ELI is bounded by the **Python scientific stack** — extremely capable, but not a dedicated CFD/FEA suite or a game engine. For a heavy specialist simulation you'd still use purpose-built software; ELI covers the enormous middle ground of scientific/engineering plotting, dynamics and modelling.

- Very large simulations run as fast as **your** machine allows — they're computed locally, offline.
- The scientific extras ship with the installer; from a source checkout, install them with `pip install -e ".[analysis]"`.

One-line examples to try

```
plot y = sin(x)/x from -20 to 20
make a 3D surface plot of a gaussian bump
simulate and animate a simple pendulum
solve the Lorenz system and show the butterfly attractor
animate Conway's Game of Life on a 50x50 grid
load model.stl, render it, and tell me its dimensions
```